Two-Link Robot — Newton-Euler Dynamics (Plant) Model

A nonlinear dynamics model of a two-link planar robot arm, derived from first principles using the Newton-Euler method, cross-checked with an independent Lagrangian derivation, and implemented as a Simulink plant model.

System Description

The system is a two-link planar arm ("pendubot"-style), with two revolute joints:

- **Link 1** is a slender rod pinned to ground at point **O**. Its absolute angle relative to the ground x-axis is $\boldsymbol{\theta_1}$.
- **Link 2** is a slender rod pinned to the end of Link 1 at point **A**. Its angle is expressed *relative* to Link 1 as **β**, so its absolute angle is $\theta_1 + \beta$.

This gives a 2-DOF planar system with generalized coordinates (θ_1, β). Both links are modeled as uniform slender rods ($I = 1/12 \cdot M \cdot L^2$), and the model accepts independent motor torques at each joint (Tm1, Tm2) as well as an arbitrary external disturbance force at the end effector (F_dx, F_dy) — so it can represent free swinging motion, actuated motion, or motion under an external load/disturbance.

Derivation Walkthrough

The equations of motion were derived by hand using the **Newton-Euler method** rather than the more common shortcut of going straight to Lagrangian mechanics. It's more work, but the internal joint reaction forces (which a Lagrangian derivation doesn't give you) fall out for free — useful if you ever need to size the joints or actuators for real loads. The full ~20-page handwritten derivation is included in this repo; what follows is a walkthrough of the key steps, not a full reproduction.

1. Kinematics: three reference frames

Before any forces are summed, the derivation sets up a chain of rotation matrices:

- **R** — the ground (inertial) frame

- **R_1** — attached to Link 1, rotated by θ_1 from ground
- **R_2** — attached to Link 2, rotated by β from R_1 (so its absolute angle is $\theta_1+\beta$)

Standard 2D rotation matrices transform positions between these frames, and are differentiated once and twice (by hand, using the product rule on the rotating unit vectors) to get closed-form velocity and acceleration expressions for any point on either link, purely in terms of θ_1, β, and their derivatives. These accelerations are what get substituted into the force/moment sums below — this is the step that makes Newton-Euler for a two-link chain more involved than a single pendulum.

2. Free Body Diagrams

Link 1 — pinned to ground at O, connected to Link 2 at A:

- Reaction forces `F1x`, `F1y` at the ground joint O
- Reaction forces `F12x`, `F12y` at joint A, exerted on Link 1 by Link 2
- Weight `M1·g`, acting at Link 1's center of mass
- Motor torque `Tm1` and joint friction `T_Ff` at O
- Reaction torque `Tm2` and friction `T_FfA`, exerted on Link 1 at A by Link 2's actuator/friction (Newton's third law)

Link 2 — pinned to Link 1 at A, free end at B:

- Reaction forces `-F12x`, `-F12y` at A (equal and opposite to Link 1's FBD, by construction)
- Weight `M2·g`, acting at Link 2's center of mass
- Any tip mass `Mp·g` and disturbance forces `Fdx`, `Fdy`, applied at the end effector B
- Motor torque `Tm2` and friction `T_FfA` at A

Alongside each FBD, an **effective force diagram** ("Effect Diagram") is drawn — the d'Alembert-style diagram showing `M·ẍ` and `M·ÿ` acting at the center of mass, and `I·θ̈` as the rotational term. Setting ΣF = (effective force) and ΣM = (effective moment) for each link is what actually produces the equations of motion.

3. Summing forces and moments

For Link 1, summing forces and taking moments about O gives:

```
ΣFy (Link 1):  -F1y + F12y - M1·g = M1·ÿ_C1

ΣM_O (Link 1): Tm1 - T_Ff + T_FfA - M1·g·Loc·cosθ1
                - F12x·L1·sinθ1 + F12y·L1·cosθ1 = I1·θ̈
```

For Link 2, summing forces and taking moments about A gives:

```
ΣFx (Link 2): -F12x + Fdx = M2·ẍ_C2
ΣFy (Link 2): -F12y - M2·g - Mp·g - Fdy = M2·ÿ_C2

ΣM_A (Link 2): Tm2 - T_FfA - M2·g·Lac·cos(β+θ1) - Mp·g·L2·cos(β+θ1)
                - Fdx·Lac·sin(β+θ1) - Fdy·L2·cos(β+θ1) = I2·θ̈2
```

where `θ̈2 = θ̈1 + β̈` since β is defined relative to Link 1.

4. Substituting the kinematics

The `ẍ_C1`, `ÿ_C1`, `ẍ_C2`, `ÿ_C2` terms above aren't independent unknowns — they're fully determined by θ_1, β, and their derivatives, from the rotation-matrix kinematics in step 1:

```
ẍ_C1 = Loc·(-cosθ1·θ̇1² - sinθ1·θ̈1)
ÿ_C1 = Loc·(-sinθ1·θ̇1² + cosθ1·θ̈1)

ẍ_C2 = L1·(-cosθ1·θ̇1² - sinθ1·θ̈1) + Lac·[cos(β+θ1)·(β̇+θ̇1)² - sin(β+θ1)·(β̈+θ̈1)]
ÿ_C2 = L1·(-sinθ1·θ̇1² + cosθ1·θ̈1) + Lac·[-sin(β+θ1)·(β̇+θ̇1)² + cos(β+θ1)·(β̈+θ̈1)]
```

Substituting these into the five equations above (plus the ΣFx equation for Link 1, not shown here — see note below) leaves **six equations that are linear in exactly six unknowns**: the four joint reaction forces and the two angular accelerations. All the θ, β, $\dot{\theta}$, and $\dot{\beta}$ terms are, at this point, just known coefficients at any given instant in the simulation.

5. Assembling the matrix form

Because the six equations are linear in `[F1x, F12x, F1y, F12y, θ̈1, β̈]`, they collect directly into a linear system `A·Y = B`, solved each timestep as `Y = A\B`. This is the exact structure implemented in the `Two_Link_F_Kinetics` MATLAB Function block.

For example, row 2 (the Link 1 ΣFy equation above) and row 3 (the Link 1 moment equation) become:

```
Row 2:  0·F1x + 0·F12x - 1·F1y + 1·F12y - (M1·Loc·cosθ1)·θ̈ + 0·β̈
         = -(M1·Loc·sinθ1·θ̇²) + M1·g

Row 3:  0·F1x - (L1·sinθ1)·F12x + 0·F1y + (L1·cosθ1)·F12y
         - (I1 + M1·Loc²)·θ̈ + 0·β̈
         = -Tm1 + T_Ff - T_FfA + Loc·M1·g·cosθ1 - Tm2
```

The remaining four rows follow the same pattern from the Link 2 equations. Once all six rows are assembled, the block solves `Y = A\B` for the full state each timestep.

> **Note on the ΣFx equation for Link 1:** the handwritten derivation consistently gives `-F1x + F12x = M1·ẍ_C1`, but the coded `A` matrix uses `+F1x + F12x` (row 1, columns 1–2 both `+1`). This is a sign convention that changed somewhere between the derivation and the implementation — worth a quick check to confirm which one is correct, since it affects the sign of the reported `F1x` reaction force output.

6. Independent check via Lagrangian mechanics

As a second correctness check, the same system was **independently re-derived using the Lagrangian (energy) method** — kinetic and potential energy for both links, the Euler-Lagrange equation, and two resulting equations of motion in θ_1 and β. This derivation was carried out separately from the Newton-Euler one and compared against it afterward: the two methods agree. Two independent derivations converging on the same equations of motion is a meaningfully stronger correctness signal than either derivation alone.

Implementation

The model is implemented in `Two_Link_Newton_Euler.slx` as a `Plant_Model` subsystem:

Inputs: `Tm1`, `Tm2` (motor torques at each joint), `F_dx`, `F_dy` (disturbance force applied at the end effector)

Outputs: `Theta1`, `Theta1_dot`, `Beta`, `Beta_dot`

Internally, the plant is split into two pieces:

- `Two_Link_F_Kinetics` (MATLAB Function block) — implements the matrix system derived above, rebuilding `A` and `B` from the current state each timestep and solving `Y = A\B`. The code comments the raw derived equation directly above its simplified, coded form, so the block can be read side-by-side with the handwritten derivation.
- `CalculateAngles` — double-integrates the accelerations into velocities and angles, and feeds viscous friction torques (`T_FFO` at the ground joint, `T_FFA` at the link-link joint) back into the force balance, so joint friction is part of the closed-loop dynamics rather than an afterthought.

The **initial condition is deliberately non-trivial**: θ_1 starts at 280° (all other states start at 0), so the model exercises genuinely nonlinear behavior from the first timestep rather than starting from a trivial equilibrium.

Parameterization

Physical parameters (link length, diameter, material density) are used to derive mass and moment of inertia for each link under a uniform slender-rod assumption. The parameter script in this repo (`Two_Link_Robot.m`) currently uses placeholder values for demonstration — it is not yet populated with measured dimensions from a physical build. The script's commented-out section also preserves the first working revision of the `A` / `B` matrix code, kept in place for traceability against the version now living in the Simulink MATLAB Function block.

Known Placeholder

The viscous friction gains (`Viscous_Friction_Constant_Needs_Set_T_FFA` and `..._T_FFO`) are currently both set to `0.005` — a placeholder value, not yet experimentally identified. This is flagged directly in the block names in the model itself.

Repository Contents

- `Two_Link_Newton_Euler.slx` — the Simulink plant model
- `Two_Link_Robot.m` — parameter definition script
- Derivation write-up (handwritten, ~20 pages) — kinematics, free body diagrams, Newton-Euler equations, simplification pass, and Lagrangian verification

Status

Measured physical parameters and a controller are still being designed and are not yet part of this repository.